

Table Of contents

- 1. Introduction..... 2
- 2. Compilation with Debugging Symbols.....3
 - Figure 1: Program compilation and directory listing..... 3
- 3.Starting GDB and Setting a Breakpoint..... 3
 - Figure 2: Launching GDB session..... 3
- 4. Mapping the .bss Segment.....4
 - Figure 3 : Inspecting binary section addresses..... 4
 - Figure 4: Printing global variable memory address..... 4
- 5.Runtime Memory Mapping..... 5
 - Figure 5: Displaying process virtual memory layout..... 5
- 6. Stack Variable.....5
 - Figure 6: Printing stack variable details in GDB..... 5
- 7. Examining Raw Stack Memory..... 6
 - Figure 7: Raw stack memory examined using the \$sp register..... 6
- 8.Heap Memory.....6
 - Figure 8: GDB output showing the heap address and stored value..... 6

Mapping Process Runtime Topography Using Low-Level Hardware Debugging Infrastructure

1. Introduction

A running program uses different areas of memory for different purposes. Global variables, local variables, and dynamically allocated variables are stored in different memory regions. In this laboratory, GDB (GNU Debugger) is used to examine the runtime memory layout of a C program. The main areas investigated are:

I. bss – stores uninitialized global/static variables.

II. Heap – stores dynamically allocated memory using malloc().

III. Stack – stores local variables and function-related information.

The purpose of this practice is to identify the virtual address boundaries of these regions and examine the values stored inside them.

2. Compilation with Debugging Symbols

First, the source code was compiled using the `-g` option.

The `-g` option adds debugging information to the executable. This allows GDB to identify variables, source-code lines, functions, and addresses.

To confirm that the executable was created: `ls`

Expected output includes: `demo` and `demo.c`

```
.2# gcc -g demo.c -o demo
root@AYUSHA:~/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4
.2# ls
README.md  demo  demo.c
```

Figure 1: Program compilation and directory listing

3. Starting GDB and Setting a Breakpoint

The compiled program was opened in GDB using: `gdb ./demo`

```
root@AYUSHA:~/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4.2# gdb ./demo
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2017 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./demo...
```

Figure 2: Launching GDB session

Inside GDB, a breakpoint was placed before the program terminates: `break 23`

Then the program was executed: `run`

The breakpoint pauses execution so that the heap and stack are still available for inspection.

4. Mapping the .bss Segment

The following command was used: info files

The .bss section was examined using the GDB command info files. The output showed that the .bss section of the executable is located from 0x000055555558010 to 0x000055555558018. The address of the global variable bss_global_var was then obtained using the command print &bss_global_var, which returned 0x55555558014. Since 0x55555558014 falls within the .bss range 0x55555558010 to 0x55555558018, this confirms that the uninitialized global variable is stored in the .bss section.

```
(gdb) info file
Symbols from "/root/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4.2/demo".
Native process:
  Using the running image of child process 4362.
  While running this, GDB does not access memory from...
Local exec file:
  '/root/S-T-5-0-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4.2/demo', file type elf64-x86-64.
Entry point: 0x555555550e0
0x000055555554350 - 0x000055555554374 is .note.gnu.build-id
0x000055555554374 - 0x000055555554390 is .interp
0x000055555554390 - 0x0000555555543b4 is .gnu.hash
0x0000555555543b8 - 0x0000555555544c0 is .dynsym
0x0000555555544c0 - 0x00005555555457b is .dynstr
0x00005555555457c - 0x000055555554592 is .gnu.version
0x000055555554598 - 0x0000555555545d8 is .gnu.version_r
0x0000555555545d8 - 0x000055555554698 is .rela.dyn
0x000055555554698 - 0x000055555554710 is .rela.plt
0x000055555555000 - 0x00005555555501b is .init
0x000055555555020 - 0x000055555555080 is .plt
0x000055555555080 - 0x000055555555090 is .plt.got
0x000055555555090 - 0x0000555555550e0 is .plt.sec
0x0000555555550e0 - 0x0000555555552ad is .text
0x0000555555552b0 - 0x0000555555552bd is .fini
0x000055555556000 - 0x000055555556062 is .rodata
0x000055555556064 - 0x000055555556098 is .eh_frame_hdr
0x000055555556098 - 0x000055555556144 is .eh_frame
0x000055555556148 - 0x000055555556178 is .note.gnu.property
0x000055555556178 - 0x000055555556198 is .note.ABI-tag
0x000055555557d98 - 0x000055555557da0 is .init_array
0x000055555557da0 - 0x000055555557da8 is .fini_array
0x000055555557da8 - 0x000055555557f98 is .dynamic
0x000055555557f98 - 0x000055555558000 is .got
0x000055555558000 - 0x000055555558010 is .data
0x000055555558010 - 0x000055555558018 is .bss
0x00007ffff7fc02a8 - 0x00007ffff7fc02cc is .note.gnu.build-id in /lib64/ld-linux-x86-64.so.2
0x00007ffff7fc02d0 - 0x00007ffff7fc0408 is .hash in /lib64/ld-linux-x86-64.so.2
0x00007ffff7fc0408 - 0x00007ffff7fc0564 is .gnu.hash in /lib64/ld-linux-x86-64.so.2
Type <RET> for more, q to quit, c to continue without paging--q
```

Figure 3 : Inspecting binary section addresses

```
(gdb) print &bss_global_var
$1 = (int *) 0x55555558014 <bss_global_var>
```

Figure 4: Printing global variable memory address

5.Runtime Memory Mapping

The runtime memory layout of the process was examined using the GDB command `info proc mappings`. The executable's writable mapping was found between `0x0000555555558000` and `0x0000555555559000`. The heap was mapped from `0x0000555555559000` to `0x0000555555557a000`. The address of `bss_global_var`, `0x555555558014`, falls inside the writable executable mapping. The heap mapping was identified by the `[heap]` label, confirming the location of dynamically allocated memory.

```
(gdb) info proc mappings
process 4362
Mapped address spaces:

Start Addr      End Addr      Size          Offset        Perms File
0x0000555555554000 0x0000555555555000 0x1000        0x0           r--p /root/S-T-5-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4.2/demo
0x0000555555555000 0x0000555555556000 0x1000        0x1000        r-xp /root/S-T-5-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4.2/demo
0x0000555555556000 0x0000555555557000 0x1000        0x2000        r--p /root/S-T-5-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4.2/demo
0x0000555555557000 0x0000555555558000 0x1000        0x3000        r--p /root/S-T-5-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4.2/demo
0x0000555555558000 0x0000555555559000 0x1000        0x3000        rw-p /root/S-T-5-0-3-C-M-D-Operating-Systems-Security-and-Networks/lab4.2/demo
0x0000555555559000 0x0000555555557a000 0x21000       0x0           rw-p [heap]
0x00007ffff7c00000 0x00007ffff7c28000 0x28000       0x0           r--p /usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7c28000 0x00007ffff7dc0000 0x198000      0x28000       r-xp /usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7dc0000 0x00007ffff7e0e000 0x4e000       0x1c0000      r--p /usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7e0e000 0x00007ffff7e12000 0x4000        0x20e000      r--p /usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7e12000 0x00007ffff7e14000 0x2000        0x212000      rw-p /usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7e14000 0x00007ffff7e21000 0xd000        0x0           rw-p
0x00007ffff7fad000 0x00007ffff7fb0000 0x3000        0x0           rw-p
0x00007ffff7fb0000 0x00007ffff7fb8000 0x2000        0x0           rw-p
0x00007ffff7fb8000 0x00007ffff7fbc000 0x4000        0x0           r--p [vvar]
```

Figure 5: Displaying process virtual memory layout

```
0x0000555555558000 0x0000555555559000 ... rw-p ... demo
0x0000555555559000 0x0000555555557a000 ... rw-p [heap]
```

6. Stack Variable

The local stack variable was examined using GDB. The command `print stack_local_var` returned the value 42, confirming that the variable was correctly initialized. The command `print &stack_local_var` returned the address `0x7fffffffdddc`. Since local variables declared inside a function are stored in the stack region, this address represents the location of `stack_local_var` within the process's stack memory.

```
(gdb) print stack_local_var
$2 = 42
(gdb) print &stack_local_var
$3 = (int *) 0x7fffffffdddc
```

Figure 6: Printing stack variable details in GDB

7. Examining Raw Stack Memory

The raw stack memory was examined using the GDB command `x/4wx $sp`. The output showed four words of memory beginning at address `0x7fffffffddd0`. The final displayed word was `0x0000002a`. This hexadecimal value is equivalent to decimal 42, which matches the value of `stack_local_var`. Therefore, the memory examination confirms that the local variable's value is present in the stack memory.

```
(gdb) x/4wx $sp
0x7fffffffddd0: 0x00000000 0x00000000 0x00000000 0x0000002a
```

Figure 7: Raw stack memory examined using the \$sp register.

8.Heap Memory

The dynamically allocated memory was examined using the pointer `heap_ptr`. The command `print heap_ptr` returned the address `0x555555559010`. The runtime memory mapping showed that the heap extends from `0x555555559000` to `0x555555557a000`, placing `heap_ptr` inside the heap region. The command `x/1wd heap_ptr` was then used to inspect the value stored at this address. GDB returned the value 99, confirming that the value assigned and was successfully stored in the dynamically allocated memory.

```
(gdb) print heap_ptr
$4 = (int *) 0x555555559010
(gdb) x/1wd heap_ptr
0x555555559010: 99
(gdb) quit
```

Figure 8: GDB output showing the heap address and stored value.